



SLB-2024

Sécurité logicielle bas niveau

Laboratoire 3

Shellcoding et obfuscation

Département TIC - orientations ISCE/S

Professeur :

Jean-Marc Bost

Assistant :

Pascal Perrenoud

17 décembre 2024

SLB2024 – labo03 – Shellcoding et obfuscation

Table des matières

1	Introduction	3
1.1	Principe du laboratoire	3
1.2	Objectifs	3
1.3	Rendu	4
1.4	Evaluation	4
1.5	Matériel nécessaire	4
2	Analyse du binaire	6
3	Analyse de Shikata Ga Nai	7
4	Analyse du shellcode	9
5	Exécution du shellcode	10
6	Obfuscations XOR et ... (BONUS)	12

1 Introduction

1.1 Principe du laboratoire

Nous allons analyser le célèbre encodeur de metasploit, "Shikata Ga Nai", puis appliquer notre propre obfuscation à notre shellcode.

Shikata Ga Nai est un encodeur polymorphe permettant d'obfusquer un binaire. Pour chacune de ses applications sur un binaire, l'output produit sera différent.

Il est composé de deux parties :

- La première, que nous appellerons **préambule**, permet de récupérer les informations nécessaires (clé de chiffrement, nombre de bytes chiffrés, adresse du premier byte chiffrés) au déchiffrement de la deuxième partie.
- La deuxième partie, que nous appellerons **shellcode**, contient les bytes chiffrés de notre binaire initial.

Lors de l'obfuscation, il est possible de faire autant d'**itérations** de chiffrement que l'on souhaite afin d'ajouter plus de complexité à notre binaire obfusqué. Le chiffrement avec plusieurs itérations fonctionne de la manière suivante :

- Lors de la 1^{ère} itération, le shellcode est chiffré et un préambule y est ajouté afin de pouvoir le déchiffrer lors de l'exécution.
- Lors de l'itération suivante le résultat de l'itération précédente est chiffré et un préambule y est ajouté afin de le déchiffrer.

1.2 Objectifs

Les objectifs de ce laboratoire sont:

- manipuler les outils de reverse Ghidra et GDB afin de désobfusquer un shellcode encodé avec Shikata Ga Nai
- chiffrer un shellcode et l'intégrer dans un loader chargé de le déchiffrer au "runtime" (code métamorphe)

1.3 Rendu

Un court rapport répondant **uniquement aux questions** posées doit être rendu sur Cyberlearn avant l'échéance indiquée dans le rendu. Vous devez également y déposer vos **sources avec les noms de fichier indiqués**.

NB : Les réponses aux questions doivent être illustrées avec les **captures d'écran** de vos tests pertinents et les informations importantes doivent y être mises en évidence et référencées explicitement dans vos explications. Les codes source des scripts et programmes rédigés pour effectuer les manipulations et répondre aux questions doivent être référencés par leur nom de fichier dans le rapport et joints en annexe.

Aucune correction du laboratoire ne sera présentée en classe. Vous pouvez prendre RV avec l'équipe enseignante, si vous avez des questions et/ou besoin d'aide.

1.4 Evaluation

Ce rapport fera l'objet d'une évaluation basée sur :

- (80% de la note) exactitude, lisibilité, efficacité et exhaustivité des réponses aux questions
- (10% de la note) complétude, efficacité et lisibilité du code source joint en annexe
- (10% de la note) la qualité du document en termes de présentation, rédaction et illustrations

1.5 Matériel nécessaire

Tous les fichiers nécessaires vous sont disponibles sur Cyberlearn :

<https://cyberlearn.hes-so.ch/mod/folder/view.php?id=1873641>

L'extension PEDDA (<https://github.com/longld/peda>) vous sera très utile pour analyser le shellcode obfusqué avec Shikata Ga Nai.

SLB2024 – labo03 – Shellcoding et obfuscation

Ce laboratoire a été testé sur une machine virtuelle Ubuntu 22.04 LTS. Il est fortement recommandé d'utiliser la machine Ubuntu 24.04 qui vous a été fournie pour les exercices et laboratoires.

2 Analyse du binaire

Le binaire *shikata.elf* a été obfusqué avec Shikata Ga Nai.

MANIPULATION 2.1

Avec ghidra, vérifier les permissions sur les segments principaux du binaire.

QUESTION 2.1

Que remarquez-vous d'inhabituel dans les permissions attribuées aux principaux segments de *shikata.elf*. Quel lien voyez-vous avec l'obfuscation Shikata Ga Nai ?

3 Analyse de Shikata Ga Nai

Utilisez ghidra et gdb pour analyser le déchiffrement du binaire.

NB : Il est recommandé d'utiliser l'extension PEDA pour gdb. La commande `context code <N>` vous permettra d'afficher *N* lignes de code assembleur du contexte actuel.

MANIPULATION 3.1

Dans gdb, exécutez les instructions jusqu'au déchiffrement des 8 premiers bytes.

QUESTION 3.1

Que font les instructions correspondant aux 8 premiers bytes (2 premiers mots) déchiffrés ? Font-elles partie du shellcode ?

MANIPULATION 3.2

Continuez l'exécution du programme en visualisant le déchiffrement mot par mot et arrêtez-vous après le dernier déchiffrement pour voir l'intégralité du shellcode déchiffré.

Aide à la manipulation 3.2

Posez un "hardware breakpoint" après la boucle de déchiffrement. Puis, retentez la même opération avec un breakpoint normal.

QUESTION 3.2

Qu'observez-vous comme différence en posant un hardware breakpoint plutôt qu'un breakpoint tout simple après la boucle de déchiffrement ? Qu'en déduisez-vous sur GDB?

QUESTION 3.3

Avec combien d'itérations de Shikata Ga Nai le binaire a-t-il été obfusqué ?

MANIPULATION 3.3

Récupérez depuis gdb le code assembleur du préambule.

QUESTION 3.4

Quel préambule avez-vous obtenu pour la première itération (code assembleur avec adresse mémoire de chaque instruction) ?

QUESTION 3.5

A quoi sert l'instruction *fnstenv* dans le préambule ? Pourquoi est-elle utilisée ?

Indice : <https://marcosvalle.github.io/re/exploit/2018/08/25/shikata-ga-nai.html>

QUESTION 3.6

A quoi sert le registre *ecx* qui est utilisé dans le préambule ?

QUESTION 3.7

Expliquez les opérations de déchiffrements sur les 2 premiers mots mémoire du shellcode (opérations de déchiffrement, instructions assembleur résultantes).

4 Analyse du shellcode

Le fichier *shellcode.asm* contient les instructions assembleurs du shellcode qui a été chiffré avec Shikata Ga Nai.

QUESTION 4.1

Expliquez pour chaque appel système, son utilité, ses arguments, leurs valeurs et comment ils lui sont passés. Aidez-vous de la « linux system call table ».

QUESTION 4.2

Que fait le shellcode ?

5 Exécution du shellcode

QUESTION 5.1

Comment pouvez-vous utiliser le shellcode *shikata.elf* ?

MANIPULATION 5.1

Exécutez le shellcode et vérifiez/montrez son fonctionnement

MANIPULATION 5.2

Le shellcode *shikata.asm* contient des bytes qui sont à éviter dans un shellcode afin de ne pas le casser lors de l'exploitation (par exemple bytes interrompant des fonctions de lecture/copie). Modifiez le code assembleur afin d'éliminer ces bytes.

AIDE À LA MANIPULATION 5.2

Les bytes que l'on ne souhaite pas retrouver sont les suivants : *0x00*, *0x09*, *0x0A*, *0x0B*, *0x0C*, *0x0D*, *0x20*.

Le script hexaCode disponible sur cyberlearn(<https://cyberlearn.hes-so.ch/mod/folder/view.php?id=1936809>) permet d'extraire les bytes d'un exécutable. Il faut néanmoins vérifier que l'output corresponde bien aux bytes affichés avec la commande *objdump -D <exécutable>*.

QUESTION 5.2

Quelles modifications avez-vous du apporter à votre shellcode ?

QUESTION 5.3

Le comportement du shellcode a-t-il dû être modifié ? Comment ?

6 Obfuscations XOR et ... (BONUS)

MANIPULATION 6.1

Réalisez un script python qui encode votre payload avec un chiffrement XOR et un autre type d'obfuscation vu en cours.

QUESTION 6.1

Quelle obfuscation du cours avez-vous choisie ? Montrez l'effet sur le code obfusqué et expliquez en quoi elle peut compliquer le reverse.

QUESTION 6.2

A quoi devez-vous faire attention lors du choix de la clé XOR ?

QUESTION 6.3

Expliquez les lignes principales du code de votre script python.